

Open in app ↗

Sign up

Sign in

Medium

 Search Write

Part 3: Optimizing Performance with the ZeRO Optimizer



The Machine Learning Alchemist

Follow

11 min read · Nov 24, 2023



In the initial parts of this series, I discussed how to handle datasets too large for a single GPU. This involved distributing the datasets across multiple systems, with each keeping a complete copy of the model. Although this approach is effective for large datasets, it fails when the size of the model itself exceeds the GPU's memory.

To overcome this challenge, a solution was required that could eliminate the inefficiencies arising from duplicating model parameters and their associated working memory. The ideal method would involve segmenting and distributing parts of the model similarly to how datasets are segmented. The ZeRO optimizer is specifically designed to address this issue by dividing both the model and its working memory, thus reducing redundancy.

ZeRO differentiates itself from previous methods with its unique structure. It is segmented into three stages of optimization, each building upon the preceding one. Notably, implementing even the early stages of ZeRO offers

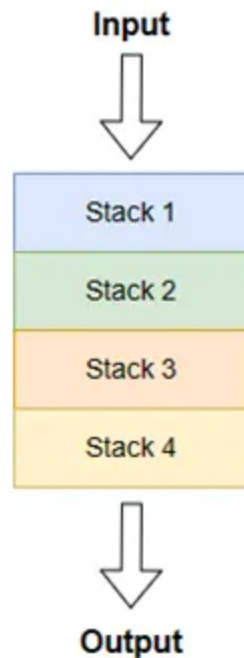
significant processing and memory advantages, without necessitating the implementation of all stages. The three progressive stages of ZeRO are as follows:

- **ZeRO-1 Optimizer State Partitioning:** In the first stage, ZeRO-1, each GPU maintains a full copy of the model and its gradients. However, the optimizer states are partitioned across GPUs. This is particularly important relevant for optimizers like Adam or LARS, which require additional memory for calculating first and second moments. While ZeRO-1 retains the full parameters and gradients, it only operates on the subset of parameters owned by each specific GPU.
- **ZeRO-2 Gradient Partitioning:** Building on ZeRO-1, the second stage involves partitioning the gradients across GPUs. Each GPU continues to hold the complete model but experiences reduced memory load for gradients.
- **ZeRO-3: Parameter Partitioning:** In the final stage, ZeRO-3 distributes the model's parameters across GPUs. Consequently, no single GPU holds the entire model, gradients, or optimizer states. This stage achieves maximum memory efficiency and allows for scaling to much larger models, as the memory requirement per GPU is significantly reduced.

Each of these levels impact various stages in the training loop. But in the explanation that follows, I'll highlight where their impact is most significant.

The Components

We begin with the basic components of our system starting with the Model:



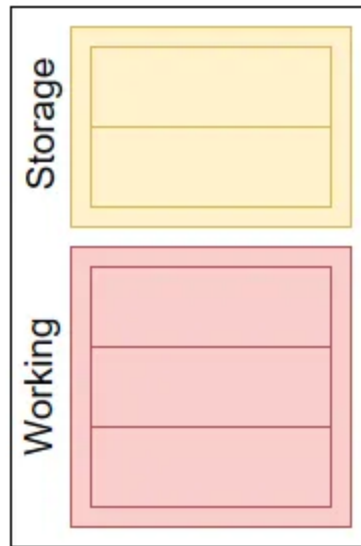
The model, a deep learning network, is configured for parallel processing using Zero Optimization. Since this example will use four systems to process the model and data, the model is divided into four stacks of layers. Input is processed from the top through stacks 1, 2, 3, and 4 consecutively until output is generated.

The second key element is the dataset:



This dataset could consist of a variety of data types, such as images, a collection of texts, or other forms of data. Mirroring the model's structure, the dataset is also segmented into four distinct partitions numbered 1 through 4. Each part will be designated to a specific system for processing.

The final component of the system are the four processing units:



The system incorporating the GPU also includes components like the CPU, memory, storage, and a network card. To simplify, we'll call this entire set of hardware a “processing unit,” or in some cases just “unit.”

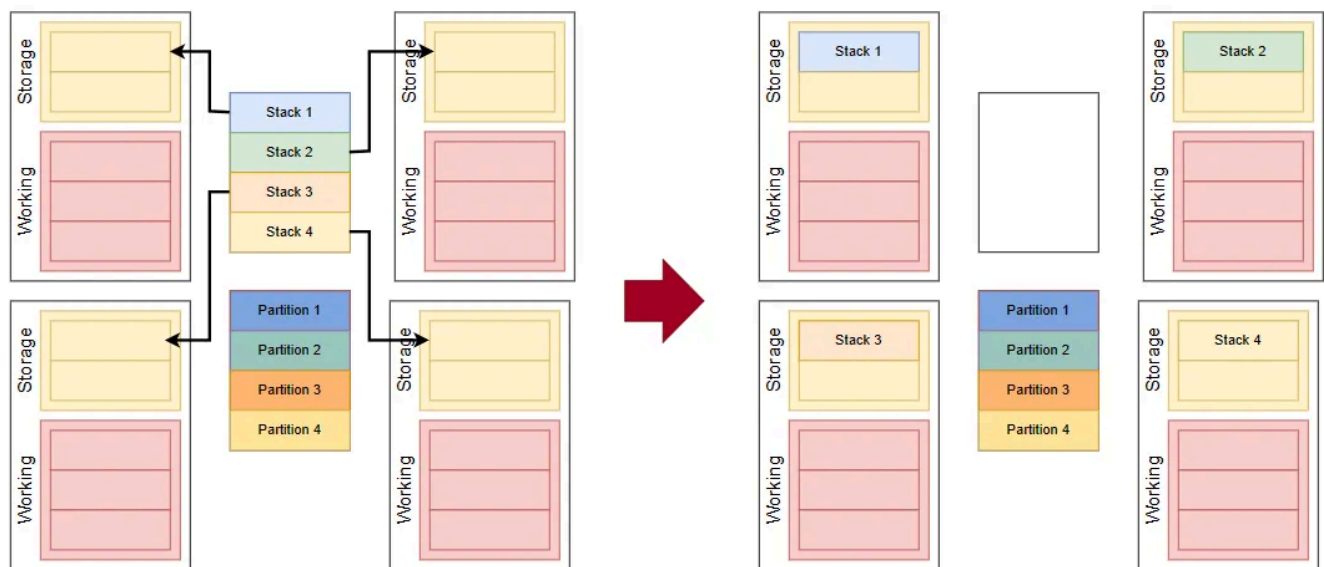
In this example, we're dealing with two conceptual elements: storage and working memory. These aren't physical things, but we use them in our explanation to make a distinction. Storage is where each processing unit keeps its portion of the model and dataset. Working memory, on the other hand, is for temporary stuff like holding model copies during processing or handling data when we're optimizing things. Even though they're not physically separate, this split helps clarify their different roles.

The Setup

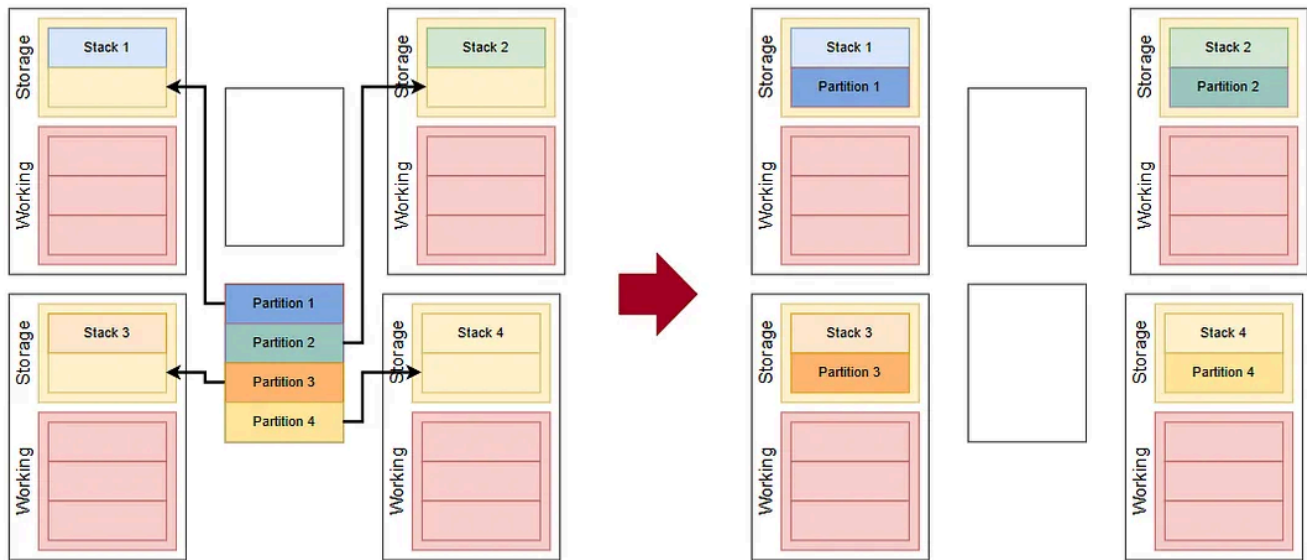
For ZeRO Optimized training to start, we need to do two key things:

First, distribute the model's layers, as stacks, to the long-term storage of each processing unit. Once done, each unit becomes the “owner” of its allocated stack.

ZeRO-3's influence is best seen right in the initial setup. Its key feature is partitioning the model parameters across several processing units. This allows us to handle really large models, as no single unit needs the memory capacity to store the entire model. But it's important to note that just partitioning the model isn't enough for processing large models. Many optimizers need extra memory, sometimes even more than the size of the model itself. We'll dive into this aspect later on.



Similarly, distribute dataset partitions to the long-term storage on each processing unit. This action assigns ownership of the respective data to each unit.



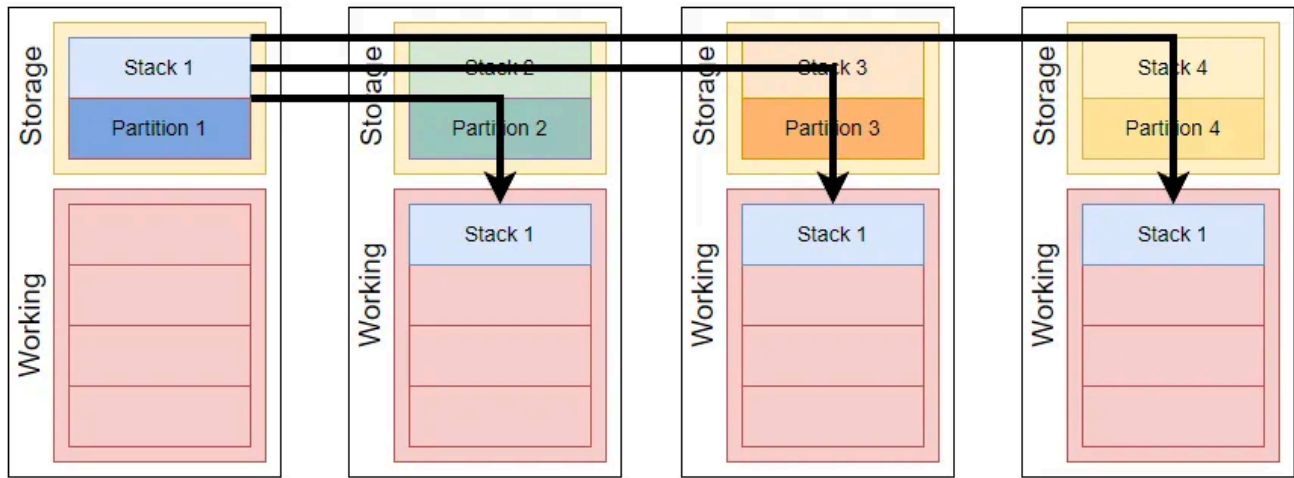
Recap:

- The deep learning model is split across layers for parallel processing.
- A dataset, comprising various data types, is segmented into four corresponding partitions, each assigned to a different processing unit.
- Model layers are distributed as stacks to each processing unit's long-term storage, assigning ownership.
- Dataset partitions are similarly allocated to the units.

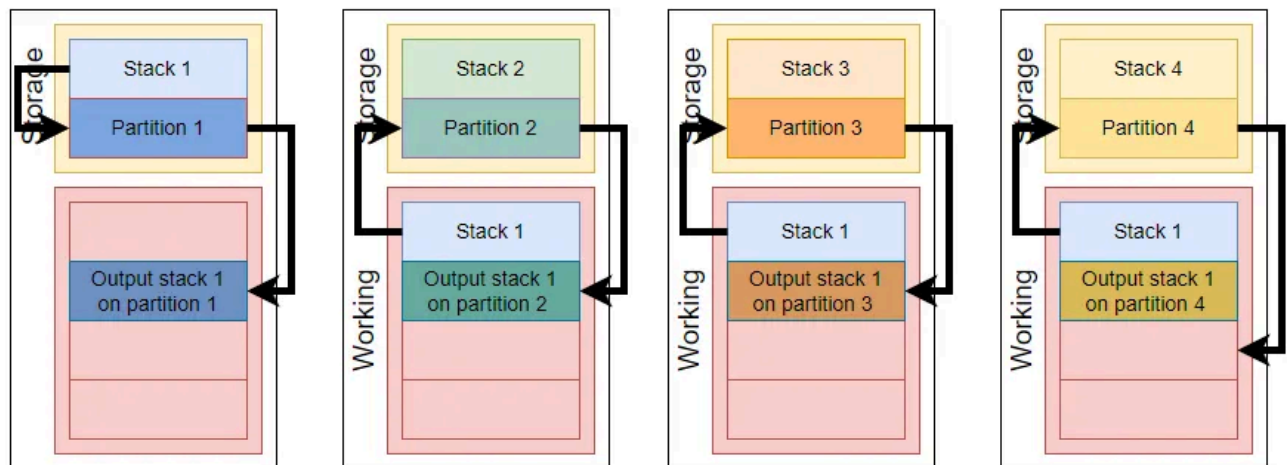
The Forward Pass :

The forward pass follows the same logic as a single GPU forward pass. The main difference is processing pauses mid-way to replicate the next layers onto a unit before continuing to the pass.

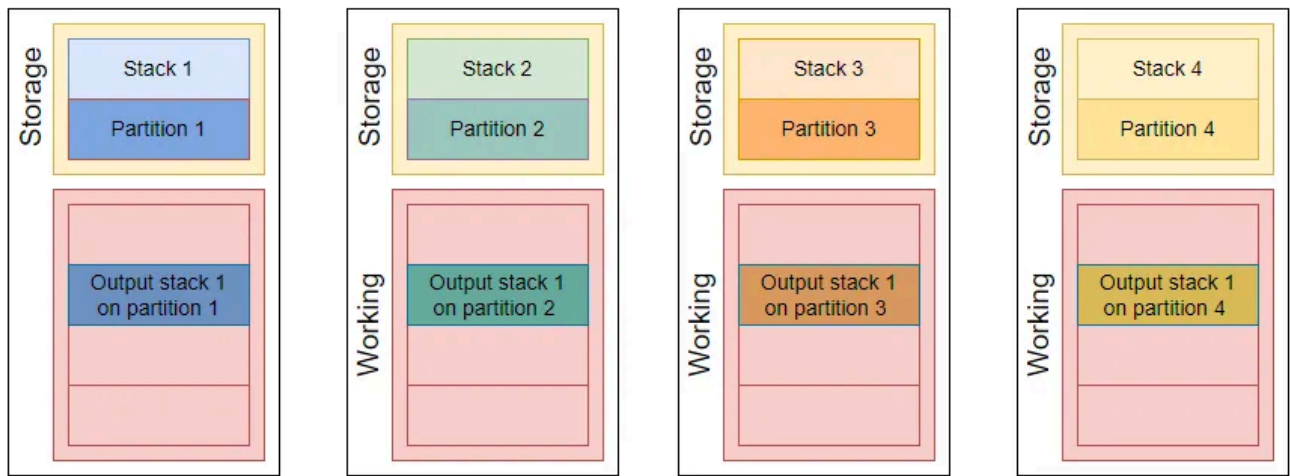
In the first step, processing unit 1 shares its model layers with all other units, which temporarily store them in working memory.



Each processing unit processes its assigned data partition using the local copy of stack 1. Consequently, all four dataset partitions are partially processed through the network's first section. The output from the last layer of stack 1 is held in each unit's working memory, where it serves as the input for processing the subsequent stack.

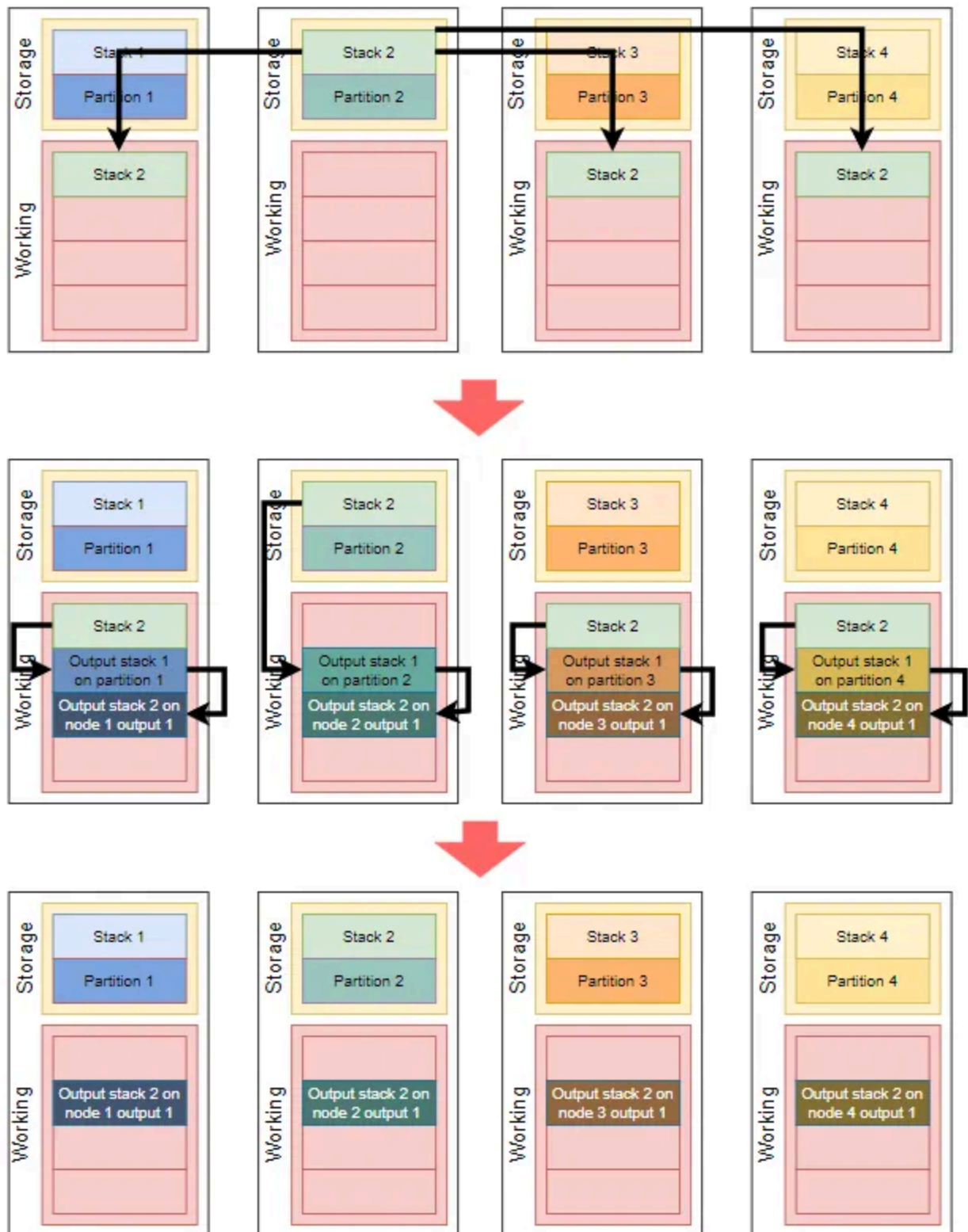


The temporary copies of stack 1 are cleared from memory, and the forward pass pauses to prepare for processing the next stack of model layers.



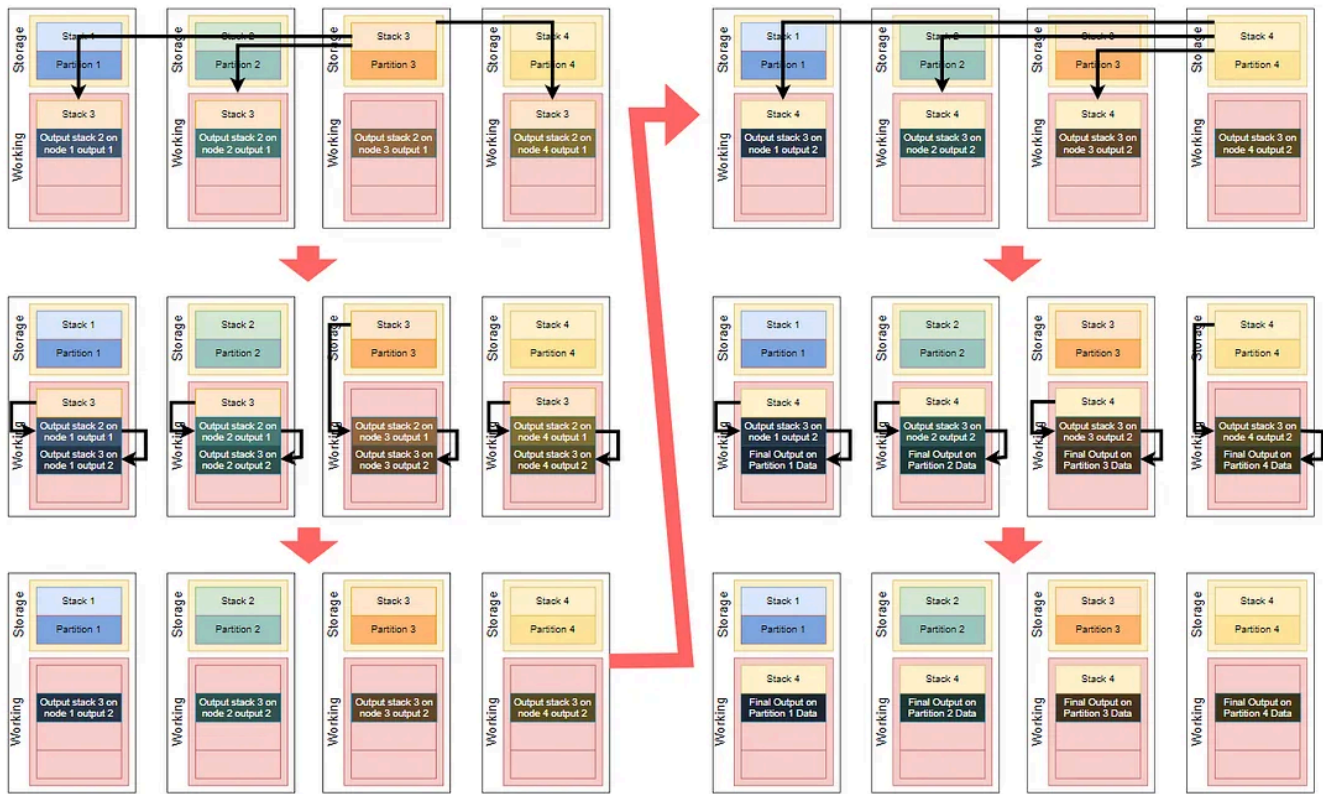
The Second Stack:

In the next step, unit 2 distributes its layers to the others. Each processing unit then uses stack 2 to process the output from the previous step, creating new output and clearing old data from working memory.

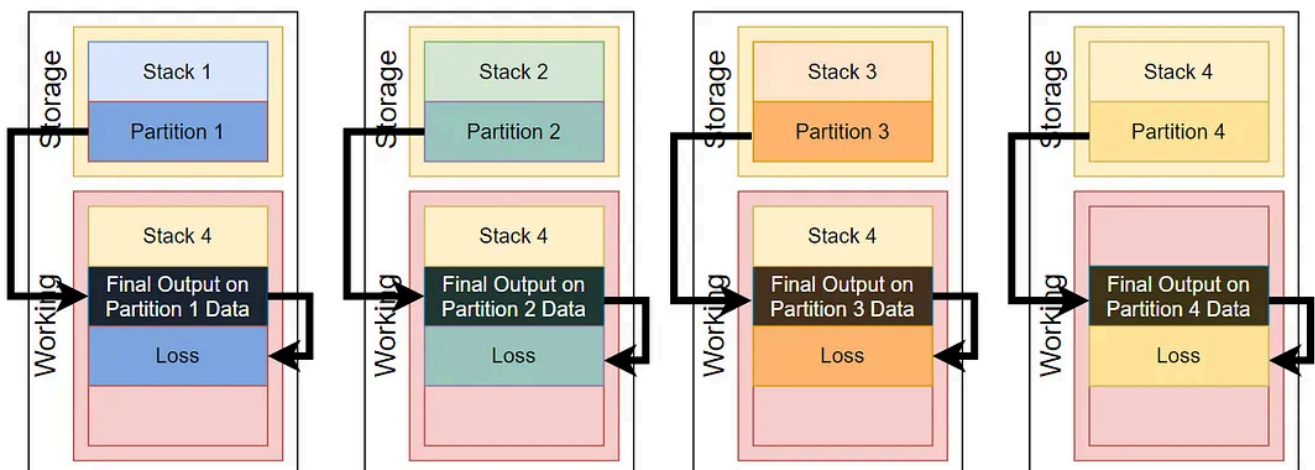


The Third and Fourth Stacks:

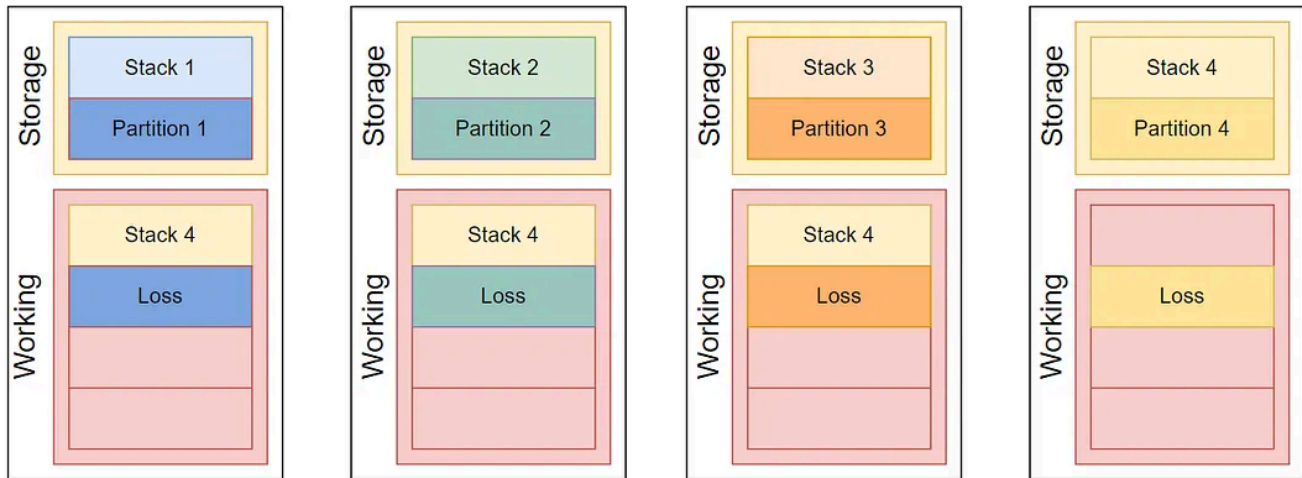
The third and fourth parts of the forward pass are processed the same as the previous steps.



Finally, the loss can be calculated using the actual values from the original partition and the final output from the forward pass at the end of step 4.



The forward pass is complete: all data partitions are processed, the working copy is removed from memory, loss is calculated using the last output and actual values from the partitions. Uniquely, the temporary copy of model stack 4 is retained for the first step of the backward pass.



Recap:

- The forward pass mirrors a single GPU process, pausing to replicate layer stacks onto subsequent units.
- Each unit processes its dataset part through the temporarily stored model layers, sequentially through stacks 1 to 4.
- As each segment completes, temporary copies are cleared, preparing for the next stack's processing.
- The forward pass ends with all partitions processed, and the final model stack 4 kept for the backward pass.

Backpropagation:

Similar to the forward pass, backpropagation spreads its workload among processing units. However, in a distinct step from the forward pass, it consolidates gradients during the final step.

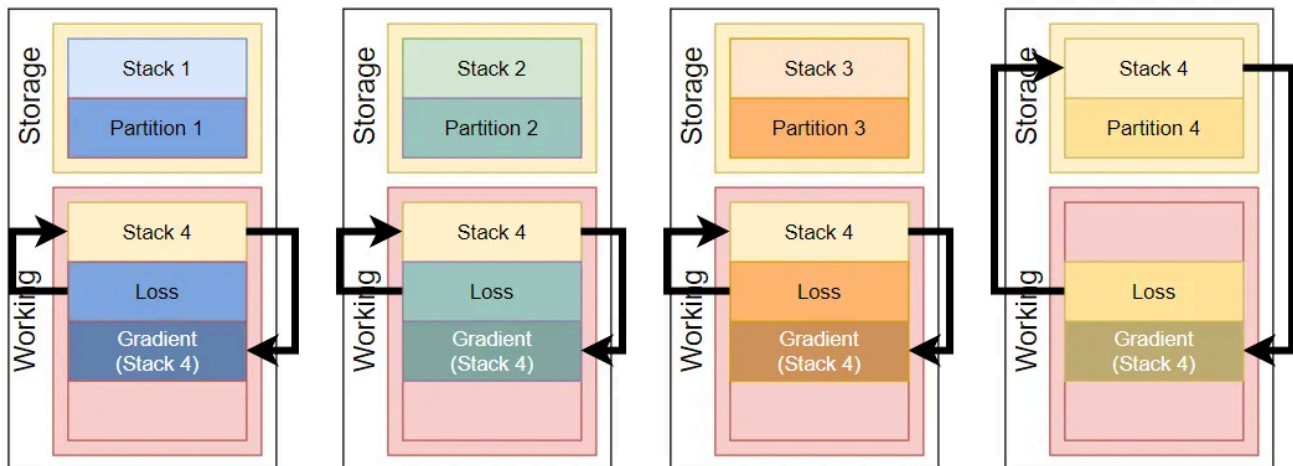
Get The Machine Learning Alchemist's stories in your inbox

Join Medium for free to get updates from this writer.

Enter your email

Subscribe

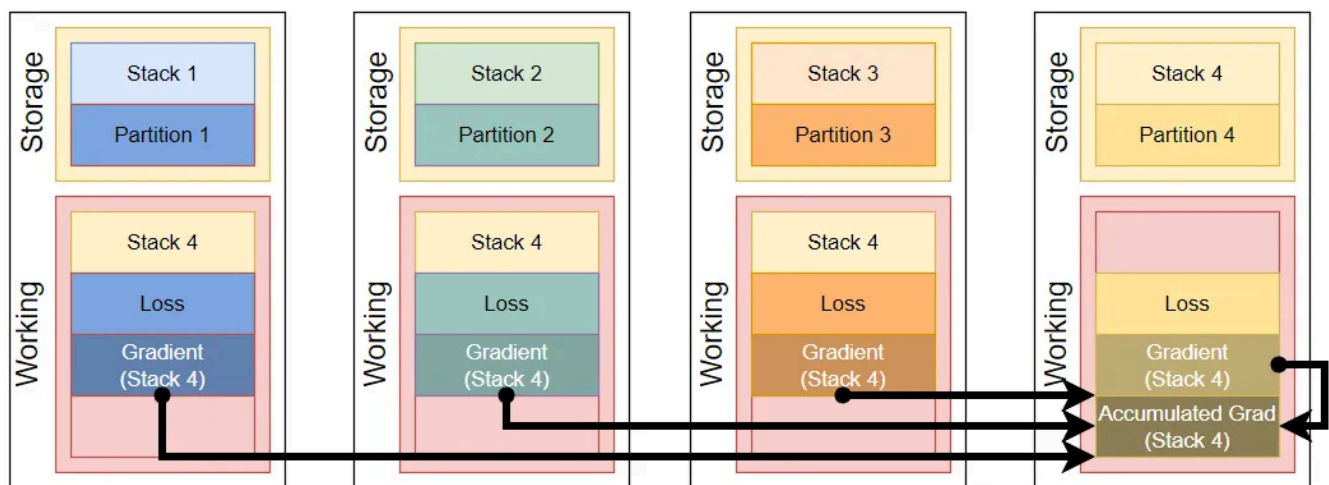
At the end of the forward pass, stack 4 is pre-staged on each processing unit. The initial step in backpropagation involves using stack 4 to calculate each node in the model's share of the loss. This is done by backpropagating the losses through the layers of stack 4. Upon completion of this step, each processing unit will have determined the gradient for its data partition as it relates to stack 4 of the model.



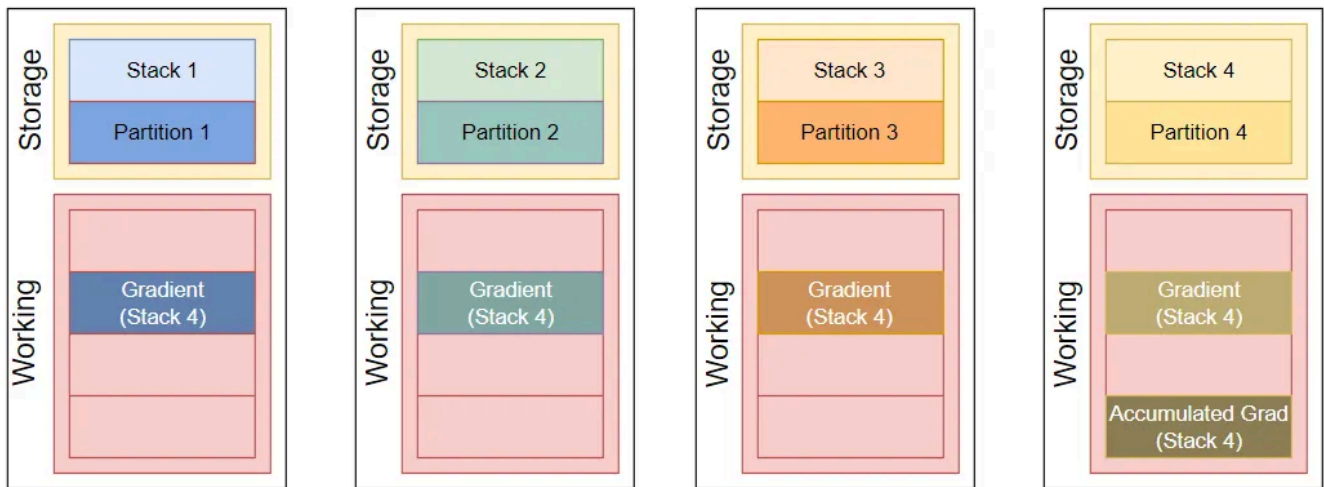
Next, every processing unit sends its calculated gradients to the unit that owns stack 4. This unit aggregates all the received gradients, forming a comprehensive set of gradients. This aggregate contains the loss attributed

to every node in stack 4, for the entire dataset processed through stack 4's section of the model.

The step where gradients are aggregated so that each processing unit holds a comprehensive gradient for its subsection of the model, rather than for the entire model, is the core of ZeRO-2. This approach is what sets ZeRO-2 apart, focusing on efficient gradient handling specific to each unit's model segment.

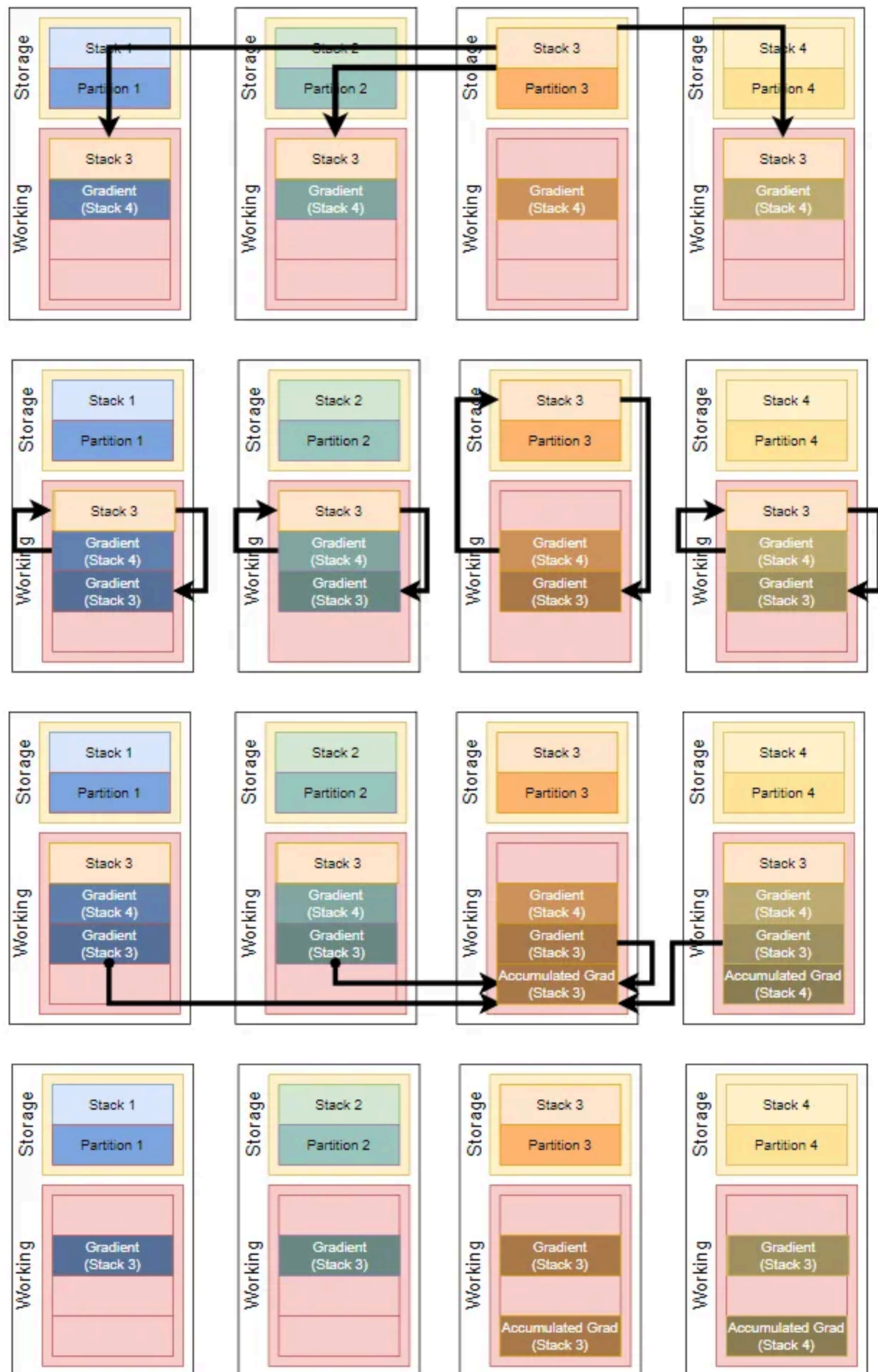


Finally, each processing unit clears its memory of the losses calculated during the forward pass and the copy of stack 4, while keeping the last set of gradient values. These retained gradients are then utilized to continue backpropagation, starting from the final layer of the model's next segment.



The Third Stack

In the previous step, due to the forward pass, stack 4 was already pre-staged on each processing unit. In this phase, stack 3 needs to be distributed to each unit. Once distributed, the gradients from the top layer of the previous stack (stack 4) are backpropagated through the layers of stack 3. This process culminates in the **accumulation** of all gradients by the owner of stack 3.



The Second and First Stacks

The backpropagation and gradients for the second and first stacks are calculated the same way as the previous steps.



The backward pass finishes with the loss fully backpropagated through the entire model. Each unit now holds gradient values for the complete dataset, corresponding to their specific segment of the model.

Recap:

- Resembling a single GPU backward pass, backpropagation with ZeRO Optimization distributes work among units.

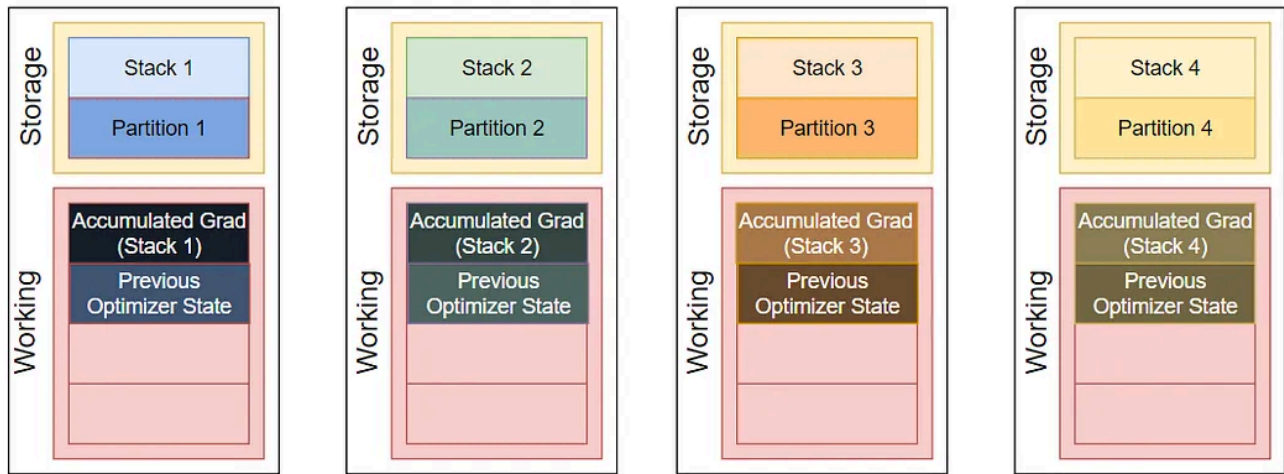
- Starting with stack 4, gradients are calculated and aggregated on the owning unit.
- Each unit clears its forward pass data but retains the last gradient values.
- Subsequent stacks follow a similar process, with gradients from previous stacks used in backpropagation and then accumulated on the owning unit.
- The process concludes with each unit holding gradient values representing the entire dataset for its model segment.

The Optimizer:

At the end of the backpropagation stage, each processing unit has its model layers and their combined gradients. An optimizer, like Adam, is then used with the gradients to update these layers. For clarity, the diagrams above depict information used by the forward and backward passes and ignore the memory that optimizers retain across iterations.

Take Adam, for example. It uses previous values to calculate the current pass's momentum and gradient variance. So, it's not just the model that needs memory; Adam needs memory for two sets of intermediate results that are both the same size as the full model. That's where ZeRO is useful. It cuts down the memory load on each unit to just the model layers it owns and the optimizer's memory state, which is only as big as those layers, not the entire model.

Including the previous optimizer state, this part of the process starts with the system in the following state:

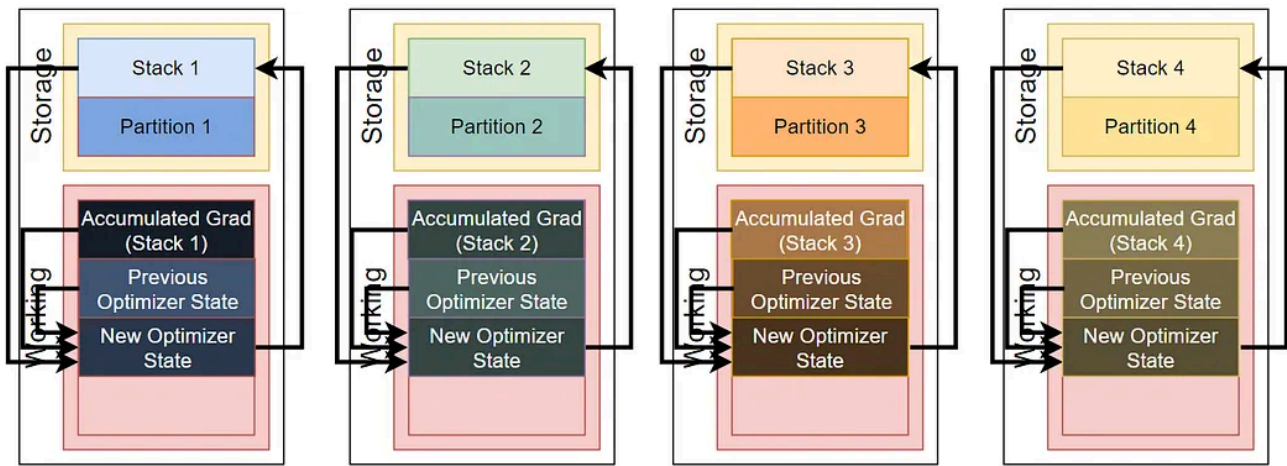


All four processing units in parallel run the optimizer algorithm. The optimizer then uses the original model stack, the accumulated gradient, and the previous optimizer state as inputs. It generates a new optimizer state as well as updated parameters for its layers of the model.

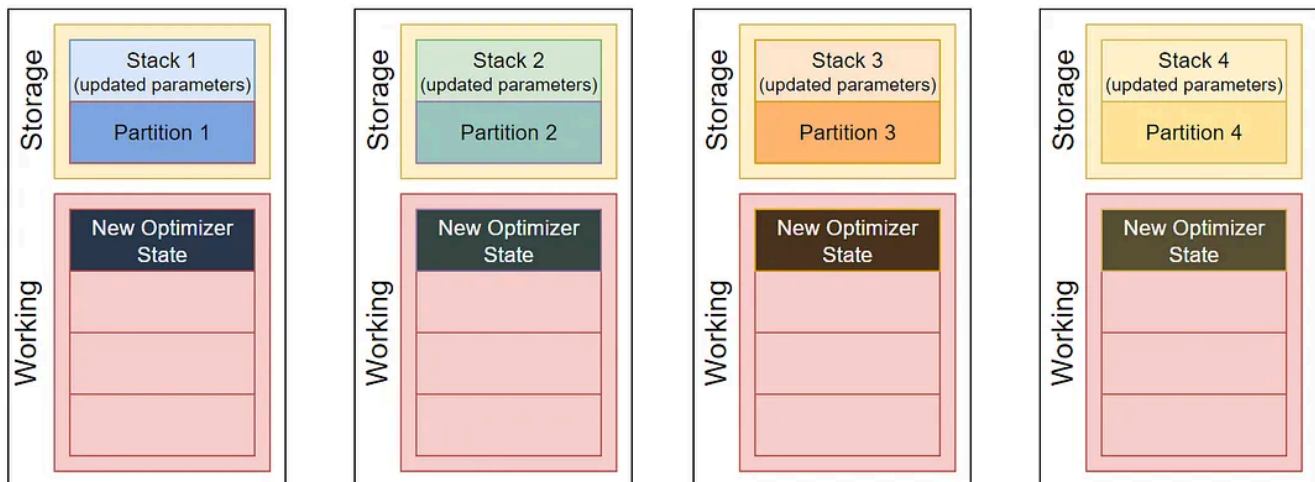
This is where ZeRO-1 makes a difference in the process. With other optimization stages, we only had a part of the model (if we just applied stage-1 we would still only designate part of the model). When using Adam — we needed extra memory, double the size of the full model. ZeRO-1 changes this by spreading this memory load across the four processing units. This means each unit only needs to handle an extra 1/2 of the model size in memory.

Including the other levels of Zero, a unit now needs just 3/4 of the full model's memory for optimization. That's a big shift from the 3x model size memory requirement in earlier algorithms. It's easy to see why is important when training incredibly large networks.

As an additional optimization that can occur here as well is the model parameters can be reduced from 32-bit floating point values to 16-bit in order to conserve even more memory.



At this point, the optimization pass is complete, and memory can be cleared of the **accumulated gradients** and the **previous optimizer states**. This leaves the updated model, the data partition, and a new optimizer state as the base to run the next forward pass.



Recap:

- After backpropagation, each processing unit has its model layers and their combined gradients ready.

- Using an optimizer like Adam, these layers are updated based on the gradients and the optimizer's state from a previous run.
- Each unit independently and simultaneously runs this optimizer on its model segment.
- The result is in a new optimizer state and updated model parameters.
- Model parameters can shift from 32-bit to 16-bit floating points to save memory.
- Upon completion, accumulated gradients and past optimizer states are cleared from memory.
- The next forward pass begins from here with the new model parameters, the unit's data partition, and a new optimizer state.

Final Thoughts:

As machine learning evolves, we're hitting the limits of handling bigger models. Distributed deep learning is becoming crucial for keeping up with this growth. Single GPU processing just can't keep pace with expanding model sizes, so distributed training isn't just a trend — it's a necessity.

Part 1: A Brief Guide to the Data Parallel Algorithm

Part 1: A Brief Guide to the Data Parallel Algorithm

In my exploration of machine learning, I quickly realized that advanced work demands far more computational power than...

[medium.com](#)

Part 2 : Scaling with the Distributed Data Parallel (DDP) Algorithm

Part 2 — Scaling with the Distributed Data Parallel (DDP) Algorithm

In the first part of this series, I explored the Data Parallel (DP) algorithm, highlighting its efficiency in scenarios...

[medium.com](#)[Machine Learning](#)[Neural Networks](#)[Deep Learning](#)[Artificial Intelligence](#)

Written by The Machine Learning Alchemist

30 followers · 31 following

[Follow](#)

Two decades in tech, Masters from Georgia Tech, Microsoft Alum. Here to demystify machine learning. Join me, and we'll discover the secrets of ML alchemy.

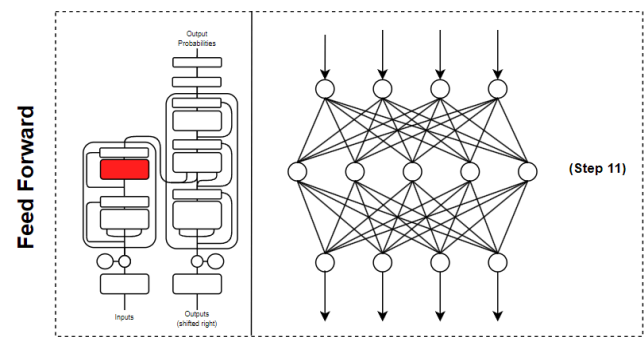
No responses yet



Write a response

What are your thoughts?

More from The Machine Learning Alchemist

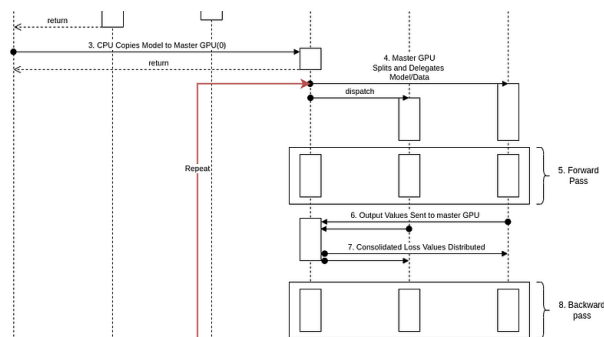


 The Machine Learning Alchemist

Decoding the Encoder

Since Google introduced the Transformer model with its self-attention mechanism, it...

Mar 29, 2024  8

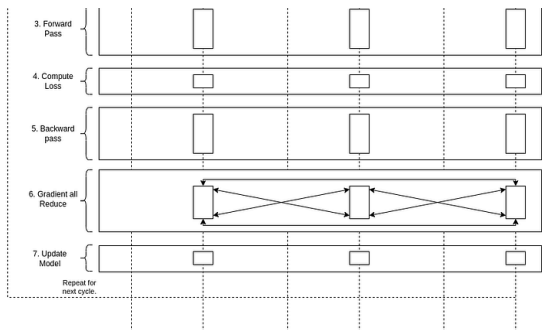


 The Machine Learning Alchemist

Part 1: A Brief Guide to the Data Parallel Algorithm

In my exploration of machine learning, I quickly realized that advanced work deman...

Nov 14, 2023  48  2



 The Machine Learning Alchemist

Part 2—Scaling with the Distributed Data Parallel (DDP)...

In the first part of this series, I explored the Data Parallel (DP) algorithm, highlighting its...

Nov 15, 2023

[See all from The Machine Learning Alchemist](#)

Recommended from Medium



50 Machine Learning Projects That Will Get You Hired in 2025 🤖



Anirban Mukherjee 🍌



50 Machine Learning Projects That Will Get You Hired in 2025 🤖

Not a member yet? Read for free here.



May 24



373



6



LLMs explained (Part 6): Smarter AI through Reinforcement Learning



In Data Science Collective by Jose Parreño

LLMs explained (Part 6): Smarter AI through Reinforcement Learning

Why fine-tuning is not enough and how reinforcement learning with human feedbac...



Jul 23



191



4



I'll Instantly Know You Used Chat Gpt If I See This



In Long. Sweet. Valuable. by Ossai Chinedum

I'll Instantly Know You Used Chat Gpt If I See This

Trust me you're not as slick as you think



Gradient Descent: ML vs Neural Networks



Oluwaseyi Akinsanya

Gradient Descent in ClassMachine Learning vs Neural Networks: Key...

By Oluwaseyi Akinsanya



May 16



17.8K



1043



6d ago



2



21 Chunking Strategies for RAG



In AI Advances by Anjolaoluwa Ajayi

21 Chunking Strategies for RAG

And how to choose the right one for your next LLM application



Jun 30



1.2K



12



Agentic AI and the Model Context Protocol: A New Era of Autonomous Agents



Anand Singh

Agentic AI and the Model Context Protocol: A New Era of...

Agentic AI—AI systems that exhibit autonomous, goal-directed behavior—is...



Jun 30



1.2K



12



Apr 23



17



1

[See more recommendations](#)